Software Architecture and Design Specification

Project: Blood Bank Management System (BBMS)

Version: 1.0

Authors: Harsimran Kaur, Chukkapalli Rohan, Dheepak Seetharam, Gautam Prasanth Nair (Section C, Team 9)

Date: 04-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	05-10-2026	Gautam Prasanth Nair	Initial draft of the Software Architecture and Design Specification, prepared from SRS v1.0 (30-09-2026) and Software Test Plan v1.0.

Approvals

Role	Name	Signature/Date
Developer	Gautam Prasanth Nair	04-10-2026
QA Lead	Dheepak Seetharam	05-10-2026
Product Owner	Harsimran Kaur	05-10-2026
Test Engineer	Chukkapalli Rohan	05-10-2026

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Blood Bank Management System (BBMS) v1.0. It turns the requirements of SRS v1.0 into a component architecture, a security architecture, sequence diagrams, an API design and an error-handling approach. It is the basis for implementation and for the Architecture and Design reference columns of the traceability matrix in the Software Test Plan.

1.2 Scope

Covers every BBMS v1.0 feature in SRS section 5: user registration and login, admin management, donor management and eligibility, search of blood stock and donors, blood donation recording, blood stock management, blood request handling and reports, together with the performance, safety, security and quality requirements of SRS section 6.
Out of scope (as in SRS 1.3): online payments, SMS or push notifications, live location

tracking, blood component separation, blood group compatibility substitution and a mobile application.

1.3 Audience
Developers, QA engineers, security reviewers, faculty and reviewers, and project team members.

1.4 Definitions
BBMS (Blood Bank Management System), SRS (Software Requirements Specification), STP (Software Test Plan), SAD (Software Architecture and Design Specification), RTM (Requirements Traceability Matrix), ADR (Architecture Decision Record), REST, JSON, RBAC (Role-Based Access Control), FEFO (First Expiry, First Out), TLS, STRIDE, CSRF, SPA (Single-Page Application), DPDP Act (Digital Personal Data Protection Act, 2023). See Appendix 5.1.

2. Document Overview

2.1 How to use this document
Provides architectural deliverables: a UML component diagram with component descriptions (3.3, 3.4), the chosen pattern with ADRs (3.5), the technology stack and data stores (3.6), risks (3.7), requirement traceability (3.8), the STRIDE threat model (3.9), three UML sequence diagrams (4.2), the API design (4.3), error handling, logging and monitoring (4.4), UX design (4.5) and open issues (4.6).
Components are identified as C1 to C17. Section 3.8 lists, for every SRS requirement, the component that implements it, the design reference and the STP test case that verifies it; the STP RTM (13.2) points to that table.

2.2 Related Documents
SRS v1.0 (02-10-2026) including Appendix C (Requirement Traceability Matrix); STP v1.0 (test cases TC-01 to TC-37 and RTM in section 13.2).

3. Architecture

3.1 Goals & Constraints

Goals (in the order of SRS 6.4: correctness and data safety first, then usability, then performance):
- Each donation, stock change and request decision is atomic, so stock and records never go out of step (NFR-QUAL-03).
- Never issue an Expired, Discarded or reserved unit; always issue the earliest-expiry unit first (NFR-SAF-01, NFR-SAF-02, BR-04).
- Protect donor and patient data with server-side RBAC, an audit trail and no personal data in logs (SO-1 to SO-4).
- Availability of at least 99% per month; 95% of searches and page loads within 3 s; 50 concurrent users; reports within 10 s (NFR-QUAL-02, NFR-PERF-01 to 04).
- Eligibility limits, shelf life and alert thresholds are configurable, not fixed in code (SRS 2.5).

- One module per feature and at least 70% unit-test coverage of business logic (NFR-QUAL-04).

Constraints: JavaScript (Node.js) as allowed by the problem statement; a relational database; HTTPS with TLS 1.2 or higher; DPDP Act 2023; latest two versions of Chrome, Firefox and Edge on screens 360 px wide and above; a team of four students and the semester schedule; the SRS 1.3 out-of-scope items are not designed.

3.2 Stakeholders & Concerns

Donors: privacy of personal data, simple phone-friendly access to their own donation history

Hospitals / Patients: fast and accurate stock search, reliable request tracking, availability in urgent cases

Blood Bank Staff: correct eligibility decisions, no unsafe issue, quick daily workflows, clear alerts

Admin: control of accounts and roles, a trustworthy audit log

Faculty / Reviewers: traceability to the SRS, quality of the design artifacts

Developers: modularity, clear APIs, testability

QA Engineers: stable interfaces, resettable test data, observable behaviour

Regulators / legal (DPDP Act): lawful handling of personal and health-related data.

3.3 Component (UML) Diagram

Figure 3.3 BBMS UML Component Diagram

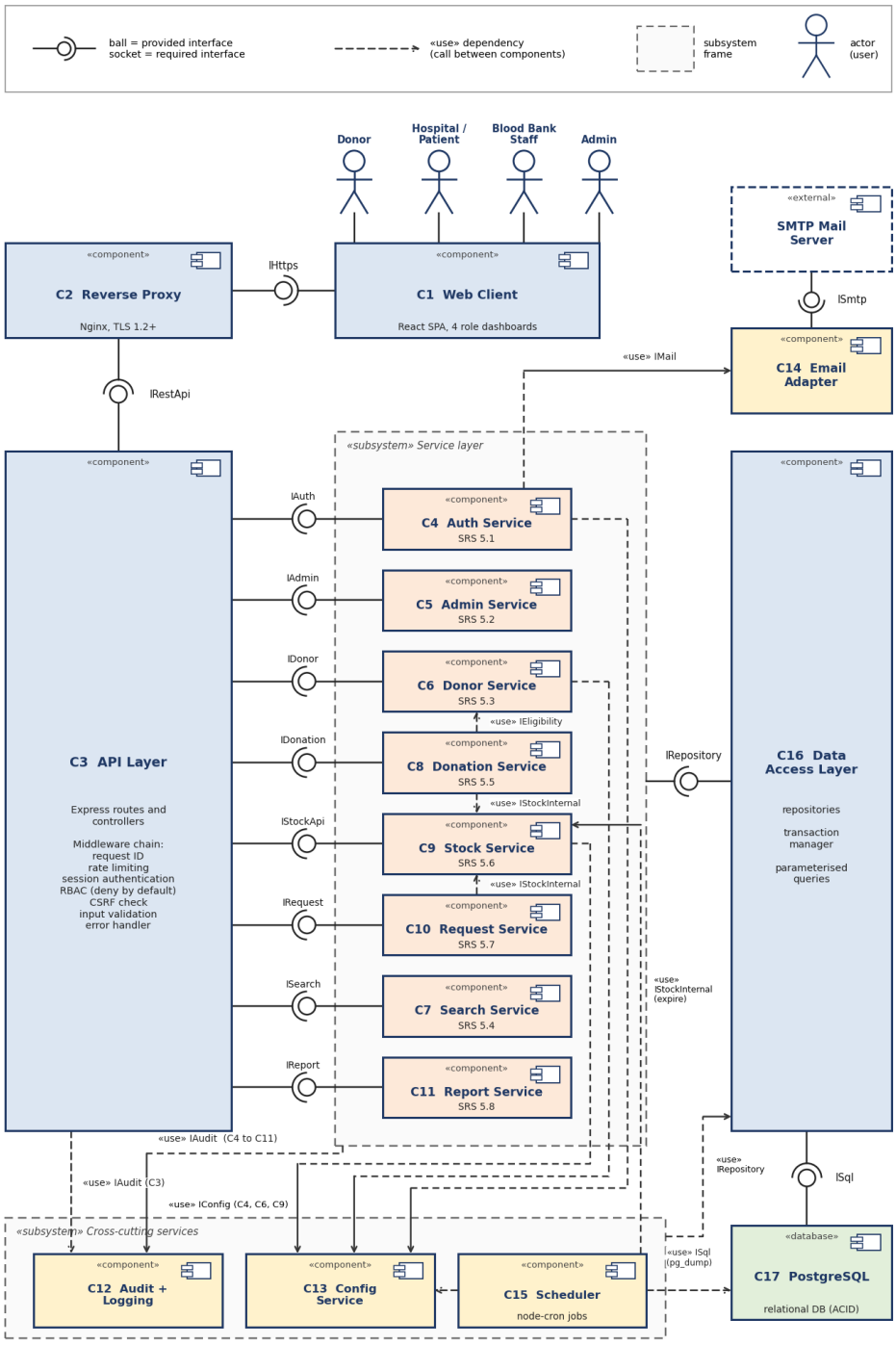


Figure 3.3 is a UML component diagram. Stick figures are the users (actors). Boxes are components («component»), except the database («database») and the external SMTP server («external»); C14 Email Adapter is drawn beside the SMTP server it wraps. A ball is a provided interface and a socket is a required interface; dashed arrows are «use»

dependencies. The service layer is drawn as one subsystem that requires IRepository as a whole (every module C4 to C11 uses it). Identifiers C1 to C17 are used in 3.4, 3.8 and the sequence diagrams. The interfaces are listed below.

Interface	Provided by	Required by
IHttps	C2 Reverse Proxy	C1 Web Client
IRestApi	C3 API Layer	C2 Reverse Proxy
IAuth, IAdmin, IDonor, IDonation, IStockApi, IRequest, ISearch, IReport	C4, C5, C6, C8, C9, C10, C7, C11 (in that order)	C3 API Layer
IEligibility	C6 Donor Service	C8 Donation Service
IStockInternal (create unit, allocate units, expire units)	C9 Stock Service	C8 Donation Service, C10 Request Service, C15 Scheduler
IRepository	C16 Data Access Layer	Service layer (C4 to C11) and C12, C13
ISql	C17 PostgreSQL	C16 Data Access Layer
IAudit	C12 Audit and Logging	C3 API Layer, service layer (C4 to C11)
IConfig	C13 Config Service	C4 Auth (lockout), C6 Donor (eligibility limits), C9 Stock (shelf life, thresholds), C15 Scheduler (job and backup settings)
IMail	C14 Email Adapter	C4 Auth Service
ISmtp	SMTP Mail Server (external)	C14 Email Adapter

3.4 Component Descriptions

- **C1 Web Client:** React single-page application with role-specific dashboards, forms that mark mandatory fields and show inline errors, and alerts shown with text as well as colour. Talks to C3 in JSON over HTTPS.
- **C2 Reverse Proxy:** Nginx. Terminates TLS 1.2+, redirects HTTP to HTTPS, serves the static client files and forwards /api requests to C3.
- **C3 API Layer:** Express routes and controllers with a middleware chain: request ID, rate limiting, session authentication, RBAC (deny by default), CSRF check, input validation and sanitising, and the standard error handler.
- **C4 Auth Service (SRS 5.1):** registration (Donor and Hospital/Patient only), credential check at login, 5-failure / 15-minute lockout (settings from C13), password policy and bcrypt hashing, one-time reset links, own profile. The session itself is created and validated by C3 (ADR-04).
- **C5 Admin Service (SRS 5.2):** staff account creation, filtered user list, activate/deactivate, role assignment, last-admin protection, audit-log view.
- **C6 Donor Service (SRS 5.3):** donor records and validation, update, deactivate/reactivate (never delete), the Eligibility Engine (age, weight, gender-based gap, limits read from C13), ineligibility reasons, donor history and donor-to-account linking.
- **C7 Search Service (SRS 5.4):** stock search over Available, unexpired units; staff-only donor search over Active and Eligible donors; pages of 20; "No matching results found"

handling.
- **C8 Donation Service (SRS 5.5):** records a donation after the eligibility check, handles the screening result, creates the donation and, when screening Passed, the blood unit; updates the donor's last donation date; 24-hour correction with audit.
- **C9 Stock Service (SRS 5.6):** unit lifecycle (Available, Reserved, Issued, Expired, Discarded), shelf life (read from C13) and expiry date, FEFO allocation, low-stock and expiring-soon alerts, discard with a mandatory reason, non-negative stock. Provides IStockApi (summary, units, discard) to C3 and IStockInternal (create unit, allocate units, expire units) to C8, C10 and C15.
- **C10 Request Service (SRS 5.7):** submit, validate, track and cancel requests; Pending queue (Urgent first, then oldest); approve with FEFO reservation; reject with reason; issue with blood group confirmation.
- **C11 Report Service (SRS 5.8):** Stock, Donation and Request reports with filters; CSV and PDF export; Staff and Admin only.
- **C12 Audit and Logging:** (IAudit) append-only audit trail (user, action, timestamp, original and modified values) written through C16 inside the transaction of the calling service and kept 1 year; application logs with passwords, phone numbers, emails and addresses redacted.
- **C13 Config Service:** reads settings (eligibility limits, shelf life 35 days, low-stock threshold 5, expiring-soon window 7 days, lockout and session settings, backup schedule and retention) from the system_config table through C16.
- **C14 Email Adapter:** sends password-reset links through SMTP; the message contains no other personal data.
- **C15 Scheduler:** in-process node-cron jobs: the midnight expiry job (FR-STK-03) and the nightly database backup (NFR-SAF-03); records the last run status for monitoring.
- **C16 Data Access Layer:** repositories that use parameterised queries, and a transaction manager so that each donation, stock change and request decision is one atomic transaction.
- **C17 PostgreSQL:** relational data store with foreign keys, UNIQUE and CHECK constraints and indexes.

3.5 Chosen Architecture Pattern and Rationale

Layered architecture implemented as a modular monolith. One deployable Node.js application is split into the layers presentation, API, service and data access, and into one module per SRS feature (route, controller, service, repository). Reasons: clear separation of concerns; per-feature modules meet NFR-QUAL-04; a single database transaction can span donation, stock and request records (NFR-QUAL-03); simple to build, deploy and test for a team of four and a load of 50 concurrent users.

Rejected: microservices, because distributed transactions between donation, stock and request would risk stock going out of step, and the operational overhead is not justified for this scale and schedule. An event-driven design with a message queue was rejected because v1.0 has no notifications. The decisions are recorded as ADRs:

ADR	Decision	Rejected alternative	Reason
ADR-01	Layered modular monolith	Microservices	Atomic transactions across donation, stock and request; simple deployment.
ADR-02	Node.js with Express, REST and JSON	Python / Django	Allowed by the problem statement; one language for client and server; Jest and Supertest are planned in the STP.
ADR-03	PostgreSQL	MySQL	Row-level locking (SELECT ... FOR UPDATE), CHECK constraints, partial indexes and pg_dump backups. MySQL stays a valid fallback (SRS 2.4).
ADR-04	Server-side sessions in an HttpOnly, Secure, SameSite cookie; 15-minute idle expiry	JWT kept in browser storage	A deactivated account can be cut off at once (FR-ADM-03); the cookie is not readable by scripts (NFR-SEC-04).
ADR-05	In-process midnight expiry job, plus an expiry filter inside every search, allocate and issue query	Job only	Even if the job is late or missed, an expired unit can never be found or issued (NFR-SAF-01).
ADR-06	Pessimistic row locking when approving a request	Optimistic retry	Two staff can never reserve the same unit; stock never goes negative (FR-STK-07).

3.6 Technology Stack & Data Stores

Client: React single-page application, responsive CSS (usable from 360 px).
Server: Node.js (LTS) with Express; express-validator for validation; helmet; express-rate-limit; express-session with a PostgreSQL session store and CSRF tokens; bcrypt; nodemailer (SMTP); node-cron; pdfkit and a CSV writer for exports; pino for logging with redaction.
Data store: PostgreSQL accessed through node-postgres (pg) with versioned migration and seed scripts.
Infrastructure: Nginx reverse proxy with TLS 1.2+; pm2 or Docker Compose to run the app; pg_dump backup every 24 hours kept for 30 days.
Test tools (STP): Jest, Supertest, Postman, JMeter.

Main tables (the six tables required by SRS 7.1 plus supporting tables):

Table	Key columns (SRS Appendix B)	Relations and constraints
users	user_id, name, email, phone, password_hash, role, status, failed_count, locked_until	email and phone UNIQUE; role in (Donor, Hospital/Patient, Staff, Admin)
donors	donor_id, full_name, date_of_birth, gender, blood_group, weight_kg, phone, email, address, city, pin_code, last_donation_date, status	phone UNIQUE; blood_group in the 8 valid groups; linked to users by phone or email
donations	donation_id, donor_id, collection_date, volume_ml, blood_group, screening_status, donation_status, recorded_by, created_at, medical_notes	FK donor_id, recorded_by; volume_ml in (350, 450)
blood_units	unit_id (UNT-XXXXX), donation_id, blood_group, volume_ml, collection_date, expiry_date, status, discard_reason	FK donation_id; status in (Available, Reserved, Issued, Expired, Discarded)
blood_requests	request_id, requester_id, patient_name, blood_group, units, required_by_date, hospital_name, contact_number, priority, status, rejection_reason, decided_by, decided_at, created_at	FK requester_id; units between 1 and 10
blood_request_units	request_id, unit_id	Links a request to its reserved units; unit_id UNIQUE
audit_log	log_id, user_id, action, entity, original_value, new_value, created_at	FK user_id; insert-only

Table	Key columns (SRS Appendix B)	Relations and constraints
system_config	key, value	Eligibility limits, shelf life, thresholds, lockout and session settings
password_reset_tokens, sessions	token hash, user_id, expires_at, used_at; session id, data, expires_at	One-time, 15-minute reset tokens; sessions with 15-minute idle expiry

Stock counts are derived by counting blood_units rows by status and blood group, with no separate counter column, so a count cannot go negative (FR-STK-07). Indexes: blood_units (blood_group, status, expiry_date), blood_requests (status, priority, created_at), audit_log (created_at, user_id) and the unique indexes on phone and email.

3.7 Risks & Mitigations

Risk	Mitigation
Two staff approve requests for the same blood group at the same time and reserve the same unit	Approval runs in one transaction with SELECT ... FOR UPDATE on the request and unit rows (SD-3).
Midnight expiry job fails, or the server is down at 00:00	Search, allocation and issue queries also test expiry_date against the current date; the job runs again at start-up; its last run is monitored.
A reserved unit expires while its request is Approved	Issuing an expired unit is always blocked (NFR-SAF-01; TC-06, TC-15). Proposed handling of the affected request is an open issue in 4.6: flag it so staff can re-allocate.
Data loss or corruption	Nightly pg_dump with 30-day retention and a restore test (TC-34); atomic transactions.
A user reads another user's data by changing an ID in the URL	Own-record filters use the session user, never an ID sent by the client; RBAC on every route; 403 and a log entry (TC-09, TC-24).
Personal data leaks through logs or error messages	pino redaction of password, phone, email and address; generic client error messages (TC-35).
Failed logins used to lock other users out	Only the targeted account locks, for 15 minutes; per-IP rate limiting (SO-3, TC-02).
Slow search with 10,000 donors and 50,000 units	Indexes, pages of 20, connection pool; JMeter test (TC-17).
Eligibility defaults differ from the official guidelines	Limits live in system_config and can change without a code change (FR-DON-04).
SMTP service unavailable	The same generic message is shown, the failure is logged and the user can request a new link; tests use a mailbox stub.
Tight schedule for a team of four	Modules per feature so work runs in parallel; high-priority features built and tested first.

3.8 Traceability to Requirements

Every SRS v1.0 requirement is mapped to the component that implements it (Architecture reference) and to the place in this document where its design is described (Design reference). The same order and IDs are used in the STP RTM (13.2) and SRS Appendix C.

Sl.	Requirement ID	Requirement	Architecture ref (component)	Design ref	Test case (STP)
1	FR-AUTH-01	Register account	C4 Auth Service; C3 validation	4.3 POST /api/auth/register	TC-01

Sl.	Requirement ID	Requirement	Architecture ref (component)	Design ref	Test case (STP)
2	FR-AUTH-02	Reject duplicate account	C4 Auth Service; C17 UNIQUE keys	4.3 POST /api/auth/register (409)	TC-19
3	FR-AUTH-03	Login by role	C4 Auth Service; C3 session; C1 dashboards	4.2 SD-1; 4.3 POST /api/auth/login	TC-19
4	FR-AUTH-04	Account lockout	C4 Auth Service; C13 Config	4.2 SD-1; 4.3 (423 ACCOUNT_LOCKED)	TC-02
5	FR-AUTH-05	Password reset	C4 Auth Service; C14 Email Adapter	4.3 forgot-password, reset-password	TC-20
6	FR-AUTH-06	Manage own profile	C4 Auth Service	4.3 GET, PUT /api/auth/me	TC-20
7	FR-AUTH-07	Logout and session end	C4 Auth Service; C3 session	4.3 POST /api/auth/logout; ADR-04	TC-18
8	FR-ADM-01	Create staff account	C5 Admin Service	4.3 POST /api/admin/users	TC-21
9	FR-ADM-02	View and filter users	C5 Admin Service	4.3 GET /api/admin/users	TC-21
10	FR-ADM-03	Activate / deactivate user	C5 Admin Service; C4 login check	4.3 PATCH .../status	TC-11
11	FR-ADM-04	Assign role	C5 Admin Service	4.3 PATCH .../role	TC-21
12	FR-ADM-05	Audit log	C12 Audit and Logging; C5	4.3 GET /api/admin/audit-logs; 4.4	TC-21
13	FR-ADM-06	Protect last admin	C5 Admin Service	4.3 PATCH .../status, .../role (409 LAST_ADMIN)	TC-11
14	FR-DON-01	Add donor record	C6 Donor Service	4.3 POST /api/donors	TC-22
15	FR-DON-02	Validate donor data	C6 Donor Service; C3 validation	4.3 POST /api/donors (400, 409)	TC-22
16	FR-DON-03	Update donor and history	C6 Donor Service; C12	4.3 PUT /api/donors/{id}	TC-23
17	FR-DON-04	Eligibility check	C6 Eligibility Engine; C13 Config	4.1 Eligibility Engine; 4.2 SD-2	TC-03
18	FR-DON-05	Show ineligibility reason	C6 Eligibility Engine	4.3 GET /api/donors/{id}; 4.1	TC-23
19	FR-DON-06	Donor views own history	C6 Donor Service; C3 own-record filter	4.3 GET /api/donors/me/donations	TC-24
20	FR-DON-07	Deactivate, not delete	C6 Donor Service	4.3 PATCH /api/donors/{id}/status	TC-25
21	FR-DON-08	Link donor to account	C6 Donor Service; C4	4.3 GET /api/donors/me/donations; 4.6	TC-24
22	FR-SRCH-01	Search stock by group	C7 Search Service	4.3 GET /api/search/stock	TC-12
23	FR-SRCH-02	Stock result details	C7 Search Service	4.3 GET /api/search/stock	TC-12
24	FR-SRCH-03	Search donors	C7 Search Service; C6 eligibility	4.3 GET /api/search/donors	TC-26
25	FR-SRCH-04	Hide donor details	C3 RBAC; C12	3.9 STRIDE; 4.3 (403)	TC-09
26	FR-SRCH-05	No result message	C7 Search Service; C1	4.3 GET /api/search/stock; 4.4	TC-12
27	FR-SRCH-06	Paginate results	C7 Search Service	4.3 GET /api/search/donors (page)	TC-26
28	FR-DNR-01	Record donation	C8 Donation Service	4.2 SD-2; 4.3 POST /api/donations	TC-04
29	FR-DNR-02	Block ineligible donor	C8 Donation Service; C6	4.2 SD-2 (422 NOT_ELIGIBLE)	TC-27
30	FR-DNR-03	Create blood unit	C8 Donation Service; C9 Stock Service	4.2 SD-2; 4.1 IDs	TC-04
31	FR-DNR-04	Failed screening	C8 Donation Service	4.2 SD-2 (Failed branch)	TC-05
32	FR-DNR-05	Update last donation	C8 Donation Service; C6	4.2 SD-2	TC-27
33	FR-DNR-06	Correct entry	C8 Donation Service; C12	4.3 PUT /api/donations/{id}; 3.9	TC-27

Sl.	Requirement ID	Requirement	Architecture ref (component)	Design ref	Test case (STP)
34	FR-STK-01	Stock by group and status	C9 Stock Service	4.3 GET /api/stock/summary	TC-28
35	FR-STK-02	Expiry date	C9 Stock Service; C13 Config	4.1 IDs; 3.6 blood_units	TC-28
36	FR-STK-03	Auto-expire units	C15 Scheduler; C9 Stock Service	3.5 ADR-05; 3.7	TC-06
37	FR-STK-04	Low-stock alert	C9 Stock Service; C13; C1	4.3 GET /api/stock/summary; 4.5	TC-07
38	FR-STK-05	Expiry alert	C9 Stock Service; C1	4.3 GET /api/stock/units; 4.5	TC-28
39	FR-STK-06	Discard unit	C9 Stock Service; C12	4.3 PATCH .../discard	TC-29
40	FR-STK-07	No negative stock	C9 Stock Service; C16; C17	3.5 ADR-06; 3.6; 4.2 SD-3	TC-29
41	FR-REQ-01	Submit request	C10 Request Service	4.3 POST /api/requests	TC-30
42	FR-REQ-02	Validate request	C10 Request Service; C3 validation	4.3 POST /api/requests (400)	TC-13
43	FR-REQ-03	Assign ID, set Pending	C10 Request Service	4.3 POST /api/requests; 4.1 request states	TC-30
44	FR-REQ-04	Pending queue order	C10 Request Service	4.3 GET /api/requests/pending	TC-14
45	FR-REQ-05	Approve and reserve	C10 Request Service; C9	4.2 SD-3; 4.3 .../approve	TC-08
46	FR-REQ-06	Reject with reason	C10 Request Service; C12	4.3 .../reject	TC-31
47	FR-REQ-07	Issue units	C10 Request Service; C9	4.3 .../issue	TC-15
48	FR-REQ-08	Track and cancel	C10 Request Service	4.3 GET /requests/mine; PATCH .../cancel	TC-30
49	FR-RPT-01	Stock report	C11 Report Service	4.3 GET /api/reports/stock	TC-10
50	FR-RPT-02	Donation report	C11 Report Service	4.3 GET /api/reports/donation	TC-32
51	FR-RPT-03	Request report	C11 Report Service	4.3 GET /api/reports/request	TC-32
52	FR-RPT-04	Filter reports	C11 Report Service	4.3 GET /api/reports/{type} (filters)	TC-32
53	FR-RPT-05	Export reports	C11 Report Service	4.3 GET /api/reports/{type}/export	TC-10
54	FR-RPT-06	Report access and empty	C11; C3 RBAC	4.3 (Staff, Admin); 4.4	TC-09
55	NFR-PERF-01	Search response time	C7; C16; C17 indexes	3.6 indexes; 4.4 monitoring	TC-17
56	NFR-PERF-02	Page load time	C1; C2	4.5; 4.4 monitoring	TC-19
57	NFR-PERF-03	Concurrent users	C3; C16 connection pool	3.6; 4.4 monitoring	TC-17
58	NFR-PERF-04	Report generation time	C11; C17	4.3 GET /api/reports/{type}; 3.6 indexes	TC-32
59	NFR-SAF-01	No unsafe issue	C9; C10	3.5 ADR-05; 4.1 unit states; 4.3 .../issue	TC-06, TC-15
60	NFR-SAF-02	Confirm group on issue	C10; C1	4.3 .../issue (confirmGroupMatch)	TC-15
61	NFR-SAF-03	Daily backup	C15 Scheduler; C17	3.6 Infrastructure; 4.4 monitoring	TC-34
62	NFR-SEC-01	Password policy and hashing	C4 Auth Service	3.9 Spoofing; 4.3 register	TC-01
63	NFR-SEC-02	Role-based access control	C3 API Layer	3.9 Elevation of privilege; 4.3 Access	TC-09
64	NFR-SEC-03	Encrypted transport	C2 Reverse Proxy	3.9 Information disclosure	TC-35
65	NFR-SEC-04	Session security	C3; C4	4.2 SD-1; 3.5 ADR-04; 3.9	TC-18
66	NFR-SEC-05	Input validation	C3; C16 parameterised queries	3.9 Tampering; 4.4	TC-16
67	NFR-SEC-06	Audit trail	C12 Audit and Logging	3.9 Repudiation; 4.4	TC-21

Sl.	Requirement ID	Requirement	Architecture ref (component)	Design ref	Test case (STP)
68	NFR-SEC-07	No personal data in logs	C12 Audit and Logging	3.9; 4.4 Logging	TC-35
69	NFR-QUAL-01	Usability	C1 Web Client	4.5 UX Design	TC-36
70	NFR-QUAL-02	Availability	C2; C15; monitoring	4.4 Monitoring; 3.7	TC-37
71	NFR-QUAL-03	Data consistency	C16 transaction manager	4.1 Transactions; 4.2 SD-2, SD-3	TC-33
72	NFR-QUAL-04	Maintainability and testability	C3 to C11 modules	3.5; 4.1 Module structure	TC-37
73	NFR-QUAL-05	Browser compatibility	C1 Web Client	4.5 UX Design	TC-36
74	BR-01	Only Staff add donors/donations; only Admin manages staff and roles	C3 RBAC; C5; C6; C8	3.9; 4.3 Access column	TC-21, TC-22
75	BR-02	Users see only their own requests / donation history	C3 own-record filter; C10; C6	4.3 /requests/mine, /donors/me/donations	TC-24, TC-30
76	BR-03	Ineligible donor cannot donate until next eligible date	C6; C8	4.1 Eligibility Engine; 4.2 SD-2	TC-03, TC-27
77	BR-04	Earliest-expiry unit issued first (FEFO)	C9; C10	4.2 SD-3; 3.5 ADR-06	TC-08
78	BR-05	Urgent requests processed before Normal	C10	4.3 GET /api/requests/pending	TC-14
79	BR-06	Only Staff approve or reject requests	C3 RBAC; C10	4.2 SD-3; 4.3 .../approve, .../reject	TC-31

3.9 Security Architecture

Trust boundaries: Internet to Nginx (HTTPS only); Nginx to the API (private network); API to PostgreSQL (private network, least-privilege database user). Roles are never taken from client input: only an Admin can create Staff or Admin accounts (BR-01), and public registration can only create Donor or Hospital/Patient accounts.

Threat Modeling (STRIDE):

Threat	BBMS example	Mitigation (component)	SRS ref
Spoofing	Guessing passwords; stolen or reused session; forged reset link	bcrypt hashes and password policy; 5-failure / 15-minute lockout; HttpOnly, Secure, SameSite session cookie with 15-minute idle expiry; one-time reset token stored hashed and valid 15 minutes (C4, C3)	NFR-SEC-01, NFR-SEC-04, FR-AUTH-04, FR-AUTH-05
Tampering	SQL injection or script in forms; changing stock or request records without permission; forged cross-site request	Server-side validation and sanitising; parameterised queries; CSRF token; RBAC on every write; atomic transactions; 24-hour donation correction window with before and after values logged (C3, C16, C12)	NFR-SEC-05, SO-2, FR-DNR-06, NFR-QUAL-03
Repudiation	A staff member denies a donation entry or a request decision	Append-only audit log with user ID, action, timestamp and original and modified values, kept 1 year and viewable only by Admin (C12)	NFR-SEC-06, FR-ADM-05, SO-4
Information disclosure	Donor details shown to hospitals; data read on the network; personal data in logs; reading another user's history	TLS 1.2+ with HTTP redirected to HTTPS; donor search and details refused with 403 for non-Staff; stock results hide donor identity; log redaction and generic errors; own-record filters from the session (C2, C3, C7, C12)	NFR-SEC-03, NFR-SEC-07, FR-SRCH-04, FR-DON-08, SO-1

Threat	BBMS example	Mitigation (component)	SRS ref
Denial of service	Flooding login or search; very large reports; lockout abuse	Per-IP rate limiting; lockout limited to one account; pages of 20; request size limits and timeouts; report date-range filters (C3, C7, C11)	SO-3, NFR-PERF-01, NFR-PERF-04
Elevation of privilege	Donor or hospital user calling staff or admin APIs; last admin removed	Deny-by-default RBAC on every API route, enforced on the server; the role changes only through Admin APIs; the last active admin cannot be deactivated or demoted (C3, C5)	NFR-SEC-02, BR-01, BR-06, FR-ADM-06

Security validation: each control above is verified by the security test cases in STP section 5.1: password policy and hashing (TC-01), lockout without locking out other users (TC-02), RBAC and 403 handling (TC-09, TC-24), TLS (TC-35), session security (TC-18), injection and script payloads (TC-16), audit trail (TC-21) and no personal data in logs (TC-35).

Security objectives: SO-1 (confidentiality) is met by RBAC, TLS and log redaction; SO-2 (integrity) by validation, parameterised queries, transactions and the audit trail; SO-3 (availability) by lockout limited to the target account, rate limiting and nightly backups; SO-4 (accountability) by named user accounts and the audit log.

4. Design

4.1 Design Overview

BBMS is designed as layered modules for separation of concerns and easier maintenance. Every request follows the same path: browser, HTTPS, Nginx, C3 middleware (session, RBAC, CSRF, validation), controller, service, repository, PostgreSQL. Controllers hold no business rules; services hold the rules; repositories hold all SQL.

Module structure: one folder per feature (auth, admin, donors, search, donations, stock, requests, reports) with routes, controller, service, repository and unit tests, plus shared folders for middleware, config, audit and database.
Eligibility Engine (C6): a donor is Eligible only if Active, aged 18 to 65 inclusive, weighing at least 45 kg, and the days since the last donation are at least 90 (Male) or 120 (Female or Other). Every failed criterion is returned as a reason together with the next eligible date. All limits come from C13. C6 is called whenever a donor record is displayed and before a donation is saved.
Blood unit states: Available to Reserved (request approved) to Issued (handed over); Available or Reserved to Expired (midnight job, when expiry date is today or earlier); Available, Reserved or Expired to Discarded (staff, with a reason).
Request states: Pending to Approved to Issued; Pending to Rejected; Pending to Cancelled. Only Staff approve, reject or issue (BR-06); only the requester cancels, and only while Pending.
Donation states: Recorded (blood unit created) or Failed Screening (no unit created, reason in medical notes).
IDs: Donor ID, Donation ID and Request ID are generated by the system; Unit ID has the form UNT-XXXXX and the expiry date is the collection date plus the configured shelf life (default 35 days).

Transactions: recording a donation, discarding a unit, approving and issuing a request each run in one database transaction together with their audit entries; the service passes the open transaction (tx) to the other modules it calls, and audit entries are written through C12 (NFR-QUAL-03, NFR-SEC-06).
Dates: all dates and the midnight job use the Asia/Kolkata time zone, so "today" is the same for search, expiry and eligibility.

4.2 UML Sequence Diagrams

Three flows specific to BBMS are shown, each starting from the user (actor): SD-1 login with lockout and session creation (including unknown email, locked, wrong password and inactive account), SD-2 recording a donation and creating a blood unit (eligibility check, passed and failed screening) and SD-3 approving a request with FEFO reservation (including request not Pending and insufficient stock). Lifelines use the component names of Figure 3.3; "Data Access + PostgreSQL" stands for C16 and C17.

Figure 4.2a SD-1 Login with lockout and session creation (FR-AUTH-03, FR-AUTH-04, NFR-SEC-04)

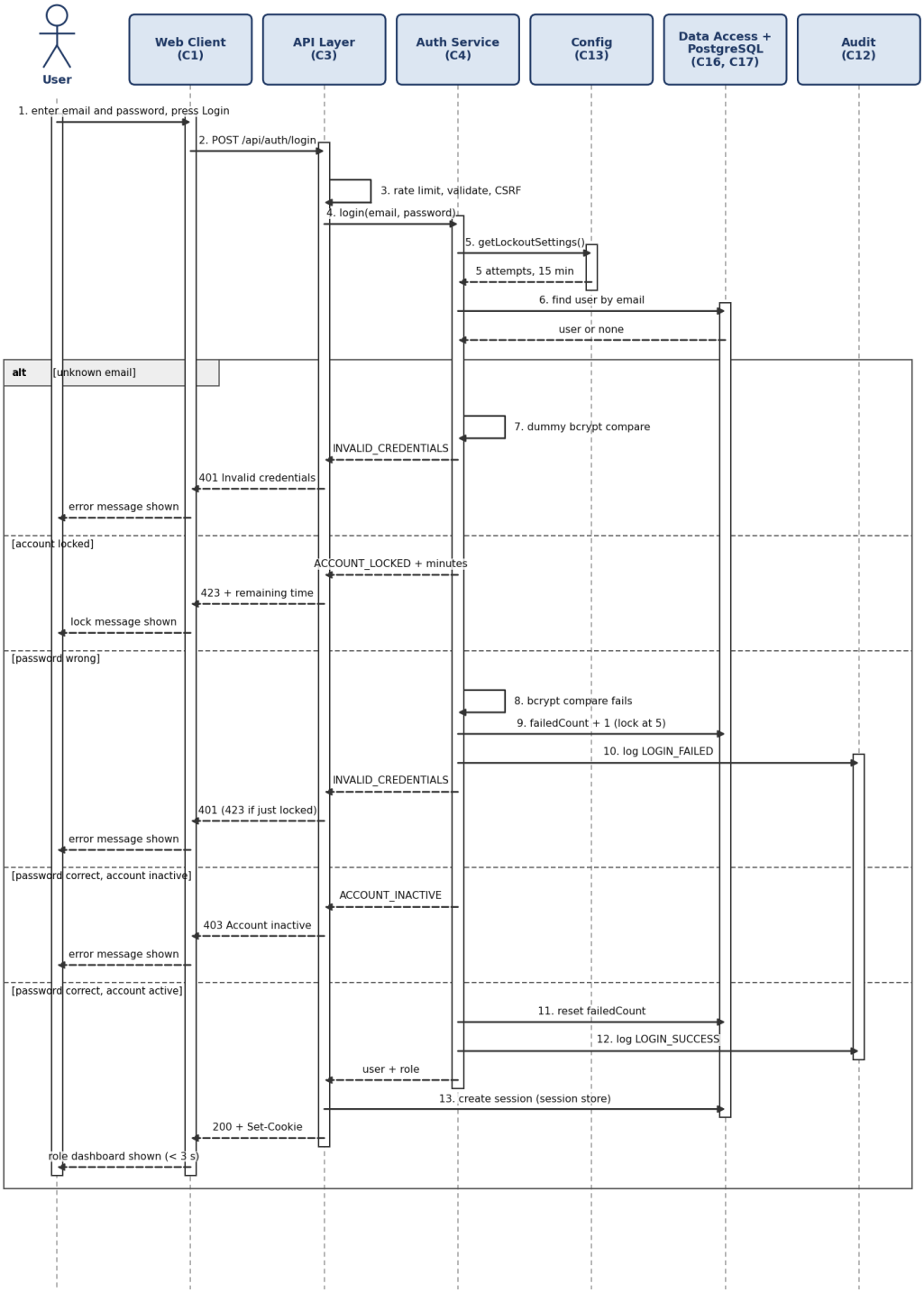


Figure 4.2b SD-2 Record a blood donation and create a blood unit (FR-DON-04, FR-DNR-01 to 05, NFR-QUAL-03)

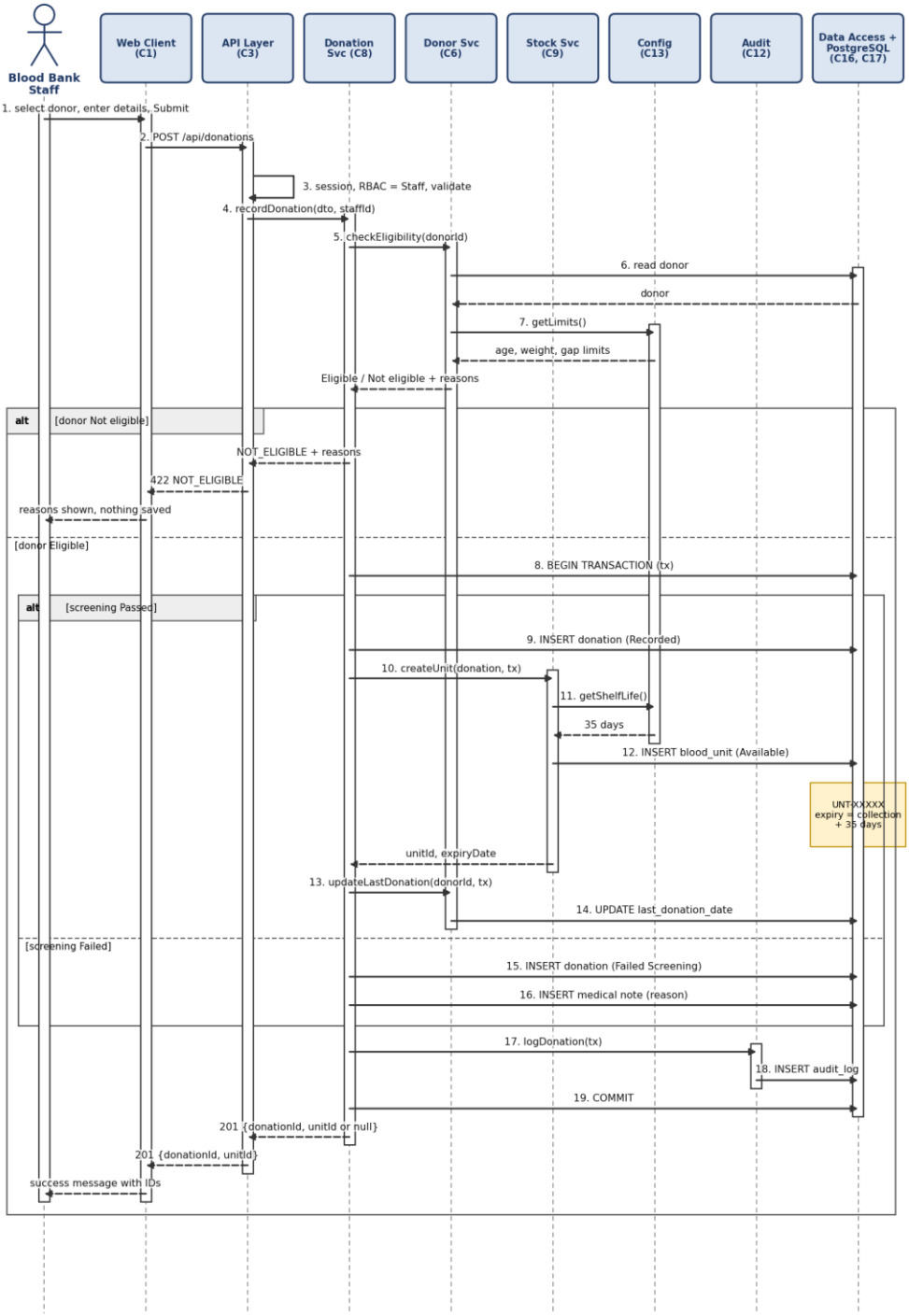
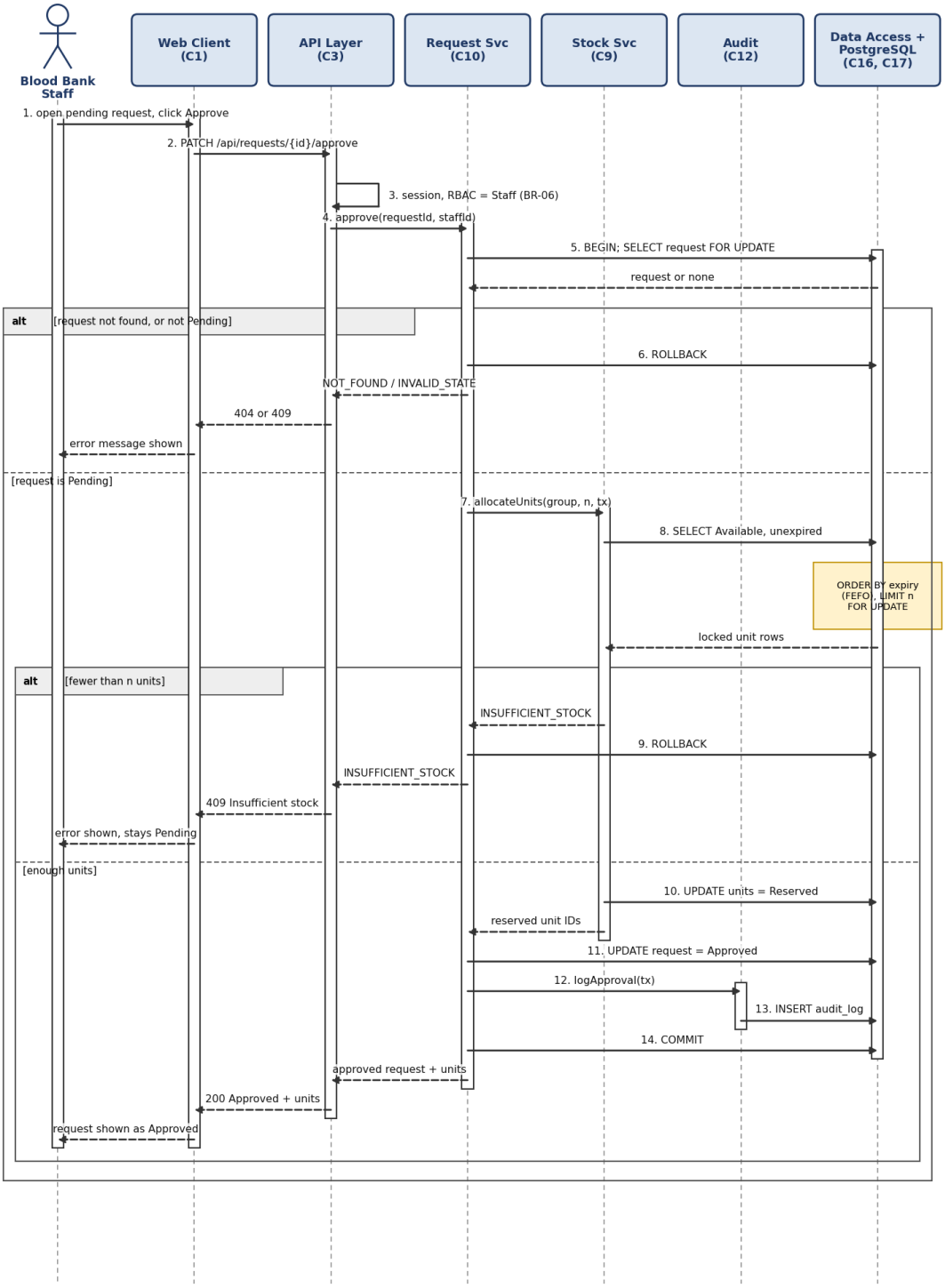


Figure 4.2c SD-3 Approve a blood request with FEFO reservation (FR-REQ-05, BR-04, FR-STK-07, NFR-QUAL-03)



4.3 API Design

Interfaces of the service components C4 to C11. Base path /api; JSON requests and responses over HTTPS; the session cookie authenticates every call except register, login and

the reset calls; write calls carry an X-CSRF-Token header. Error bodies follow section 4.4. Detailed definitions of the main endpoints:

Endpoint: /api/auth/register
Method: POST (public)
Request: {name, email, phone, password, role} (role: Donor or Hospital/Patient)
Response: 201 {userId, role, message}
Errors: 400 VALIDATION_ERROR (field messages, weak password), 409 ACCOUNT_EXISTS "Account already exists".

Endpoint: /api/auth/login
Method: POST (public)
Request: {email, password}
Response: 200 {userId, name, role}; sets the session cookie
Errors: 401 INVALID_CREDENTIALS (unknown email or wrong password, same message), 403 ACCOUNT_INACTIVE (only after the correct password), 423 ACCOUNT_LOCKED {minutesLeft} (checked first, so even the correct password is refused during the lock, TC-02).

Endpoint: /api/donors
Method: POST (Staff)
Request: {fullName, dateOfBirth, gender, bloodGroup, weightKg, phone, city, pinCode, email?, address?}
Response: 201 {donorId, status: "Active", eligibility}
Errors: 400 VALIDATION_ERROR, 403 ACCESS_DENIED, 409 DONOR_EXISTS "Donor already exists".

Endpoint: /api/donors/{donorId}
Method: GET (Staff)
Request: none
Response: 200 {donor fields, eligibility: {eligible, reasons[], nextEligibleDate}, donations[]}
Errors: 403 ACCESS_DENIED, 404 NOT_FOUND.

Endpoint: /api/search/stock?bloodGroup=O%2B
Method: GET (Hospital/Patient, Staff)
Request: query bloodGroup (one of the 8 groups)
Response: 200 {bloodGroup, availableUnits, earliestExpiry, units[{unitId, expiryDate}]}; units[] only for Staff; no donor identity. No match returns 200 with the message "No matching results found" and the criteria.
Errors: 400 INVALID_BLOOD_GROUP, 401 UNAUTHENTICATED.

Endpoint: /api/donations
Method: POST (Staff)
Request: {donorId, collectionDate, volumeMl (350 or 450), screeningStatus (Passed or Failed), medicalNotes?}

Response: 201 {donationId, donationStatus, unitId, expiryDate}; unitId and expiryDate are null when screening Failed
Errors: 400 VALIDATION_ERROR, 403 ACCESS_DENIED, 404 NOT_FOUND, 422 NOT_ELIGIBLE {reasons[]}.

Endpoint: /api/requests
Method: POST (Hospital/Patient)
Request: {patientName, bloodGroup, units (1 to 10), requiredByDate, hospitalName, contactNumber (10 digits), priority (Normal or Urgent)}
Response: 201 {requestId, status: "Pending"}
Errors: 400 VALIDATION_ERROR (past date, units out of range, bad contact number), 403 ACCESS_DENIED.

Endpoint: /api/requests/{requestId}/approve
Method: PATCH (Staff)
Request: none
Response: 200 {requestId, status: "Approved", reservedUnits[]}
Errors: 403 ACCESS_DENIED, 404 NOT_FOUND, 409 INVALID_STATE (not Pending), 409 INSUFFICIENT_STOCK.

Endpoint: /api/requests/{requestId}/issue
Method: PATCH (Staff)
Request: {confirmGroupMatch: true}
Response: 200 {requestId, status: "Issued"}
Errors: 400 VALIDATION_ERROR (confirmation missing), 409 INVALID_STATE (not Approved), 422 GROUP_MISMATCH, 422 UNSAFE_UNIT (Expired, Discarded or reserved for another request).

Endpoint: /api/requests/{requestId}/reject
Method: PATCH (Staff)
Request: {reason} (mandatory)
Response: 200 {requestId, status: "Rejected"}; the reason is shown to the requester
Errors: 400 VALIDATION_ERROR (blank reason), 403 ACCESS_DENIED (requester or other roles, BR-06), 409 INVALID_STATE (not Pending).

Endpoint: /api/donations/{donationId}
Method: PUT (Staff)
Request: {collectionDate?, volumeMl?, screeningStatus?, medicalNotes?}
Response: 200 {donationId, updated fields}; stock is updated if volume or blood group changed; original and new values are written to the audit trail
Errors: 400 VALIDATION_ERROR, 403 ACCESS_DENIED, 404 NOT_FOUND, 409 EDIT_WINDOW_EXPIRED (more than 24 hours after creation).

Endpoint: /api/stock/units/{unitId}/discard
Method: PATCH (Staff)

Request: {reason} (Expired, Hemolyzed, Contaminated, Leakage or Test Positive)
Response: 200 {unitId, status: "Discarded"}
Errors: 400 VALIDATION_ERROR (reason missing or not in the list), 403 ACCESS_DENIED, 404 NOT_FOUND, 409 INVALID_STATE (already Issued or Discarded).

All endpoints:

Component	Method and path	Access	SRS requirements
C4 Auth	POST /api/auth/register	Public	FR-AUTH-01, 02
C4 Auth	POST /api/auth/login	Public	FR-AUTH-03, 04
C4 Auth	POST /api/auth/logout	Any logged-in user	FR-AUTH-07
C4 Auth	POST /api/auth/forgot-password, POST /api/auth/reset-password	Public	FR-AUTH-05
C4 Auth	GET, PUT /api/auth/me	Donor, Hospital/Patient	FR-AUTH-06
C5 Admin	POST /api/admin/users	Admin	FR-ADM-01
C5 Admin	GET /api/admin/users?role=&status=&q=	Admin	FR-ADM-02
C5 Admin	PATCH /api/admin/users/{id}/status	Admin	FR-ADM-03, 06
C5 Admin	PATCH /api/admin/users/{id}/role	Admin	FR-ADM-04, 06
C5 Admin	GET /api/admin/audit-logs	Admin	FR-ADM-05
C6 Donor	POST /api/donors	Staff	FR-DON-01, 02
C6 Donor	GET /api/donors/{id}	Staff	FR-DON-04, 05
C6 Donor	PUT /api/donors/{id}	Staff	FR-DON-03
C6 Donor	PATCH /api/donors/{id}/status	Staff	FR-DON-07
C6 Donor	GET /api/donors/me/donations	Donor	FR-DON-06, 08
C7 Search	GET /api/search/stock	Hospital/Patient, Staff	FR-SRCH-01, 02, 05
C7 Search	GET /api/search/donors?bloodGroup=&city=&pin=&page=	Staff	FR-SRCH-03 to 06
C8 Donation	POST /api/donations	Staff	FR-DNR-01 to 05
C8 Donation	PUT /api/donations/{id} (within 24 h)	Staff	FR-DNR-06
C9 Stock	GET /api/stock/summary	Staff	FR-STK-01, 04
C9 Stock	GET /api/stock/units?bloodGroup=	Staff	FR-STK-01, 02, 05
C9 Stock	PATCH /api/stock/units/{unitId}/discard	Staff	FR-STK-06, 07
C10 Request	POST /api/requests	Hospital/Patient	FR-REQ-01 to 03
C10 Request	GET /api/requests/mine	Hospital/Patient	FR-REQ-08
C10 Request	PATCH /api/requests/{id}/cancel	Hospital/Patient	FR-REQ-08
C10 Request	GET /api/requests/pending	Staff	FR-REQ-04
C10 Request	PATCH /api/requests/{id}/approve, /reject, /issue	Staff	FR-REQ-05, 06, 07
C11 Report	GET /api/reports/{stock|donation|request}?from=&to=&bloodGroup=&status=	Staff, Admin	FR-RPT-01 to 04, 06
C11 Report	GET /api/reports/{type}/export?format=csv|pdf	Staff, Admin	FR-RPT-05

The expiry job (FR-STK-03) and the backup job (NFR-SAF-03) are run by C15 and have no public endpoint.

4.4 Error Handling, Logging & Monitoring

Error handling. Every error is returned in one format: {error: {code, message, fields[], requestId}}. Messages are the ones written in the SRS ("Account already exists", "Donor already exists", "No matching results found", "No donations recorded yet", "No records found", "access denied"). Empty results are never errors. Unexpected faults return 500 INTERNAL_ERROR with a generic message and a requestId; the stack trace stays in the server log only.

Error code	HTTP	Used when
VALIDATION_ERROR	400	A field is invalid; fields[] names each field and its message.
UNAUTHENTICATED, INVALID_CREDENTIALS	401	No valid session, session expired, or wrong email or password.
ACCESS_DENIED, ACCOUNT_INACTIVE	403	The role may not use the endpoint, or the account is deactivated. The attempt is logged.
NOT_FOUND	404	The record does not exist or is not visible to this user.
ACCOUNT_EXISTS, DONOR_EXISTS, INVALID_STATE, INSUFFICIENT_STOCK, LAST_ADMIN, EDIT_WINDOW_EXPIRED	409	The action conflicts with the current data or state.
NOT_ELIGIBLE, GROUP_MISMATCH, UNSAFE_UNIT	422	A safety or eligibility rule blocks the action; reasons[] explain why.
ACCOUNT_LOCKED	423	Five failed logins; minutesLeft is returned.
RATE_LIMITED	429	Too many requests from one address.
INTERNAL_ERROR	500	Unexpected server fault.

Logging. Structured JSON application logs with a requestId on every line. Passwords, phone numbers, email addresses and addresses are redacted and never written to logs or shown in error messages (NFR-SEC-07). The audit trail in the database records logins (successful and failed), role changes, donation entries and corrections, stock changes and request decisions with user ID and timestamp, kept for 1 year (NFR-SEC-06). Application logs and the audit trail are separate.

Monitoring. A /api/health endpoint (application and database) is checked every minute to measure the 99% availability target. Metrics: API error rate, 95th percentile response time (3 s target), failed logins and lockouts, count of 403 responses, last run and result of the expiry job and the backup job, and the Low Stock and Expiring Soon counts. An alert is raised when a job fails or the health check fails.

4.5 UX Design

The user interface follows SRS 3.1 and the quality requirements NFR-QUAL-01 and NFR-QUAL-05. It is a responsive web application (React) that works on the latest two versions of Chrome, Firefox and Edge and is usable on screens 360 pixels wide and above. After login each role sees its own dashboard (Donor, Hospital/Patient, Staff, Admin). Every screen shows the application name, the logged-in user, a logout button and a navigation menu that lists only the functions allowed for the role. Forms mark mandatory fields and show a clear

error message next to each invalid field (for example donor weight, phone number, PIN code and request units). Alerts on the staff dashboard (low stock, Expiring Soon) use a distinct colour together with text, for example "Low Stock Alert: O-", so colour is not the only signal (TC-07, TC-28). Search screens show "No matching results found" with the criteria searched, and the donor history page shows "No donations recorded yet" when empty. Issuing a request asks staff to confirm the blood group match, and a discard needs a reason from the list. The donor registration and donation recording flow is kept short so that a first-time staff user can finish it within 5 minutes using only on-screen guidance (TC-36). Detailed screen layouts will follow in the implementation sprints.

4.6 Open Issues & Next Steps

Open issues:

- Confirm the Node.js and PostgreSQL decisions (ADR-02, ADR-03) with the whole team; the STP already assumes Node.js.
- Decide what happens to an Approved request whose Reserved unit expires or is discarded: the proposed design flags the request for staff to re-allocate.
- Confirm the eligibility defaults and the 35-day shelf life against the applicable guidelines (SRS 2.6).
- Confirm the exact DPDP Act 2023 obligations (consent, retention, deletion) with the team (SRS 7.2).
- Define how a donor user is linked when the phone number or email differs between the user and the donor record (FR-DON-08).

Next steps: set up the repository and the database migrations and seed data; build the modules by priority (5.1, 5.3, 5.5, 5.6, 5.7 first); fill the Architecture and Design reference columns of the STP RTM from section 3.8; write the OpenAPI (Swagger) file from section 4.3; plan sprints.

Future enhancements (SRS 7.3): email or SMS notifications, donation appointment booking and camp management, automatic blood group compatibility substitution, mobile application.

5. Appendices

5.1 Glossary: BBMS (Blood Bank Management System); SRS, STP, SAD, RTM (see 1.4); ADR (Architecture Decision Record); REST / JSON (web API style and message format); RBAC (role-based access control); FEFO (First Expiry, First Out); TLS (encrypted web transport); STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege); CSRF (cross-site request forgery); SPA (single-page application); Blood unit (one collected bag of blood with a unique Unit ID); Eligibility (whether a donor meets the age, weight and donation-gap rules); DPDP Act (Digital Personal Data Protection Act, 2023, India).

5.2 References: IEEE 42010, OWASP (Top 10 and ASVS), NIST SP 800-160, IEEE Std 830-1998 and ISO/IEC/IEEE 29148:2018, BBMS SRS v1.0, BBMS STP v1.0, Digital Personal Data Protection Act 2023 (India).

5.3 Tools: PlantUML and draw.io (to maintain the diagrams), Swagger / OpenAPI, Postman, Jest, Supertest, JMeter, Git and GitHub.